

THE UNIVERSITY OF CHICAGO
DATA SCIENCE INSTITUTE

Agentic Skills

Motivation: why skills

Without skills

- Complex tasks require giant system prompts
- Hard to maintain, test, reuse
- Custom prompt written from scratch for every agent
- Preferences re-explained in every conversation



With skills

- Modular chunks of context: loaded, stacked, versioned independently
- A library of reusable behaviors mixed like ingredients
- Teach the agent once; reuse everywhere

summarize results | run simulation | review paper

What are skills?

- **Definition:** a documented pattern made available to an agent, giving it a reference to run code, fill out templates, or accomplish tasks
- **Origin:** defined by an open specification (agentskills.io), originally introduced in October 2025 by Anthropic
- **Structure:** a directory with at least a `SKILL.md` file
  - optionally scripts, references, templates, or even other skills

What makes skills useful?

1. **Documentation of a task**
   Reference material for commands and specs, plus the exact steps to complete the task

2. **Freedom with direction**
   The agent is still the controller, so it can troubleshoot and interpret

3. **Distilled from traces**
   Capture the steps that worked, so future runs skip trial and error

4. **Consistent results**
   Same behavior across sessions and users; portable across agents

Skills and MCP

MCP (connectivity)

- Provide live, authenticated connections to external services
- Execute actions: they cross the agent boundary
- Need a running service, credentials, and network access
- Scoped to a specific integration and dependent on its availability

Skills (knowledge)

- Carry knowledge: reasoning, API docs, schemas
- Shape how the agent thinks and constructs actions
- Embedded in the agent: no running services needed
- Portable, composable, reusable; consistent and available offline

Analogy: MCP is the professional kitchen (tools and ingredients); skills are the recipes.

Anatomy of a skill

```
my-skill/
├── SKILL.md          # Required
├── scripts/          # Optional
├── references/       # Optional
└── assets/           # Optional
```

- `SKILL.md`: required; instructions plus metadata
- `scripts/`: executable code the agent can run
- `references/`: documentation loaded only when needed
- `assets/`: templates, images, and config files used in output
- Folder name in kebab-case, matching the skill `name`

Progressive disclosure

Level 1	**Frontmatter** **Always loaded in the system prompt** Just enough to know when the skill applies
Level 2	**SKILL.md body** **Loaded when the agent judges the skill relevant** Keep under 500 lines (~5,000 tokens)
Level 3	**Linked files (scripts/, references/, assets/)** **Opened only when needed** e.g. "Read references/api-errors.md if the API returns a non-200 status"

Result: minimal token use while keeping specialized expertise

SKILL.md frontmatter

```
---
name: molecule-summarizer
description: Summarizes DFT simulation
  results for collaborators. Use when
  the user shares xTB output files or
  asks to report calculation results
license: MIT
---
```

name
kebab-case; matches the folder name — required

description
what + when to use it; < 1024 chars — required

license, compatibility, metadata
environment requirements, author, version — optional

- The frontmatter is how the agent decides whether to load the skill
- Include phrases a user might say and relevant file types

SKILL.md body

- **Role:** sets tone, expertise level, and perspective
- **Output format:** the structure you want back: headers, labels, field names, ordering
- **Rules:** constraints and guardrails; what to always do and what to never do
- **Examples:** one or two input/output pairs; the biggest single quality lever
- Order matters: role, then format, then rules, then examples (earlier content is weighted more heavily)

Example: SKILL.md

```
---
name: molecule-summarizer
description: Summarizes DFT simulation results for collaborators. Use when
the user shares xTB output files or asks to report calculation results
---
## Role
You are a computational chemist explaining results to an experimental
colleague.
Be precise but avoid jargon where plain language works just as well.
## Output Format
For every molecule report:
**Molecule:** {name or SMILES}
**Energy:** {value} Hartree
**Key Properties:** {2-3 bullet points}
**Interpretation:** {1-2 sentences: what does this mean chemically?}
**Caveats:** {anything the experimentalist should know before trusting this
result}
```


THE UNIVERSITY OF CHICAGO
DATA SCIENCE INSTITUTE

Example: SKILL.md

```
## Rules
- Always include caveats: never present a computational result as ground truth
- If the optimization did not converge, say so prominently before anything else
- Round energies to 6 decimal places
## Example
Input: xTB optimization completed for caffeine,
energy = -40.123456789 Hartree, HOMO = -0.34 eV
Output:
**Molecule:** Caffeine
**Energy:** -40.123457 Hartree
**Key Properties:**
- HOMO energy: -0.34 eV
- Geometry converged in 24 steps
- No imaginary frequencies
**Interpretation:** Stable minimum on the potential energy surface.
HOMO energy suggests moderate electron donor character.
**Caveats:** GFN2-xTB is a semiempirical method: suitable for screening  but
verify key results with DFT before drawing experimental conclusions.
```

Writing a good description

✔ Good: what, when, triggers

"Gyrokinetics simulation code GS2. Use when installing GS2, writing or modifying GS2 input namelists, configuring species/grid/field/collision parameters, or launching GS2 runs on local machines or HPC clusters."

✘ Bad: vague / missing triggers

- "Helps with chemistry." (too vague)
- "Creates sophisticated multi-page scientific reports." (no triggers)
- "Implements PBS resource selection syntax with select, ncpus, ngpus chunks." (too technical, no user triggers)

Formula: **What it does** + **When to use it** + **Key capabilities**

Best practices: what goes in a skill

- **Start from real expertise:** extract from a hands-on task, or synthesize from your runbooks, specs, and past failures, not from generic LLM knowledge
- **Add what the agent lacks, omit what it knows:** project conventions, edge cases, the specific tools to use
- **Moderate detail:** concise, stepwise guidance with a working example beats exhaustive documentation
- **Coherent units:** like a function, one skill encapsulates one coherent job
- **Progressive disclosure:** keep `SKILL.md` lean and move detail into `references/`

Limitations of skills

- **Context cost:** every loaded skill consumes tokens before the conversation starts
- **No guarantee of compliance:** the model reads a skill as a strong suggestion, not a hard constraint
- **Conflicts:** contradictory skills produce unpredictable behavior
- **Maintenance:** stale skills are worse than no skills
- **Hard to test:** needs an eval set, not eyeballing a few responses
- **Prompt injection:** user input or tool results in the same context can override a skill

The Genesis Mission Skills Repository

- Hosted on OSTI GitLab
- Enables broader access to and sharing of DOE-relevant skills
- Helps prevent many copies of the same skills
- Does not require specialty installation
- Important: Read a skill before installing it (you are running a shared skill)



gitlab.osti.gov/genesis/genesis-skills

Activity 2: Create a skill

1. **Pick a task**
   Brainstorm and decide on a computation or task relevant to your domain.
   - Pick something you've done several times before and know the "gotchas"
   - Sanity check: can the coding agent already do a good job without the skill?

2. **Create the skill**
   Create `.agents/skills/` in the project root and put your skill folder with `SKILL.md` inside.
   - Write a high level workflow and have your coding agent help write the skill

3. **Execute**
   Add the skill to your coding agent and run the task

References

[1] **Anthropic guide to building skills for Claude:**

https://resources.anthropic.com/hubfs/The-Complete-Guide-to-Building-Skill-for-Claude.pdf

[2] **Open standard specification:**

https://agentskills.io/home

[3] **Genesis Mission Skills Repository:**

https://gitlab.osti.gov/genesis/genesis-skills

Resource: Setup Pi Coding Agent



Why Pi?

- Lightweight coding agent, customizable
- Highly compatible with open-source models
- Claude Code is tuned for Anthropic models
- OpenCode and Hermes add context overhead that hurts shorter-context models

Set up Pi with the ALCF Inference Service

Install Pi

```
$ curl -fsSL https://pi.dev/install.sh |
sh
```

Point Pi at the ALCF Inference Service

```
$ alcf-ai agent configure pi
```

Resource: Pi skills documentation

Layout

```
your-skill/
├── SKILL.md
├── scripts/
│   └── your_script.sh
├── references/
│   └── formats.md
└── assets/
    └── template.json
```

Reference

- Pi documentation on skills:

  pi.dev/docs/latest/skills